Testing

Nothing in this file has been run. There is no test suite, no `pyproject.toml`, and no application code yet — these are the checks the project requires, not evidence of passing results.

Required Before Completion

- [ ] Relevant tests pass, or each failure is explained.
- [ ] Typecheck and lint pass.
- [ ] User-visible changes were checked in a browser (Chromium/Chrome primary flow, Firefox smoke) when applicable.
- [ ] The affected behaviour was re-checked after code changes or new evidence.
- [ ] No test was skipped or weakened without human approval; no oracle or expected outcome was edited to obtain a green result.
- [ ] Ordinary host/CI tests executed no binary.
- [ ] Evidence is reported in the final response, separating what was run from what is proposed.

Commands

Every command below is **unverified** — none has been run in this project.

- All tests: `.venv/bin/python -m pytest`
- Single test: `.venv/bin/python -m pytest tests/unit/test_acceptance.py::test_not_run_cannot_pass`
- Typecheck: `.venv/bin/python -m mypy app`
- Lint/format: `.venv/bin/python -m ruff check .`
- Build: none — there is no frontend compile step and no packaging command has been chosen.
- Browser/device check: manual flow in Chromium/Chrome, Firefox smoke test; Playwright Python is planned for the critical journey once the app exists.
- Real binary checks: **not available in CI.** They require the disposable Ubuntu VM with its guest agent and snapshot controls.

Keep actual binary execution out of ordinary host/CI tests. CI can cover schema, state transitions, lint, typing, and mocked-worker behaviour; anything that parses, compiles, runs, or symbolically analyses a binary belongs in the guest.

What To Test

Change type	Minimum check
Pure logic (acceptance rules, hashing, state transitions)	Unit test
API/data flow, artifact registration, migrations	Integration test with mocked workers
Contract, acceptance engine, outcome semantics	Unit + negative-case tests; unknown/not_run/timeout must not pass

Change type	Minimum check
UI behaviour and honest status labels	Browser check of the three views, including an unfinished and a failed stage
Broker, isolation, session, origin, CSRF, export paths	Human review plus focused tests; verify traversal/symlink rejection
Route A / Route B integration	Guest-side integration test on the frozen public fixture
Symbolic verifier	Calibration against the original violation and a deliberately defective candidate
AI/tool behaviour	Prompt/tool evaluation plus data-boundary checks
Auth, billing, migrations, deployment	Not applicable — no accounts, billing, or deployment exist in this MVP. Migrations still need a human-reviewed test.

Required Test Groups

The Tech Design specifies these groups. They are the acceptance surface, not suggestions.

Group	Required scenarios
Acceptance engine	Required `unknown`/`not_run` cannot pass; a failed regression rejects; `cancelled` and `error` stay distinct; both routes are needed for MVP acceptance
Identity/state	Wrong hash or stale approval rejected; duplicate start is idempotent; contract edits invalidate approval; a restart cannot accept partial output
Repair integration	Wrong architecture; unexpected bytes; no safe placement; malformed candidate; unresolved relocation; failed Route B round trip
Security verifier	Detects the original violation and a defective candidate; boundary and off-by-one cases; unknown/timeout/errored/unconstrained states remain non-success
Behaviour	Empty, smallest, typical, and maximum valid input; first invalid input and a longer one; the oracle covers required return/output values and side effects
AI integration	Empty or invalid output; wrong identity; provider failure; budget exhaustion; injection strings; unauthorized confidential transfer
Local boundary	Missing session; cross-origin POST; traversal/symlink export; malicious HTML output; guest credential and network isolation
GUI	Setup → review → start → compare → export; responsive during work; cancellation; browser refresh mid-run; unfinished stages visible
Replay	An independent teammate repeats both routes' validation from frozen candidates and manifests, without original source

Concrete Flows To Verify

These are the user-visible journeys worth exercising once the app exists.

1. **Setup rejects a bad input before running anything.** Upload a wrong-architecture ELF (for example an ARM or 32-bit build) and a truncated file. Each produces an actionable message and no run starts.
2. **Identity is recorded and bound.** After upload, the job shows the SHA-256 of the original. Approving with a stale contract revision or a mismatched input hash returns 409 and starts nothing.
3. **Review gate actually gates.** An unapproved contract cannot start a run; the analyst can edit the contract, and the edit creates a new revision and invalidates the previous approval.
4. **Both routes run from the same original.** Both attempts reference the identical original hash and the same contract revision, and neither modifies the original bytes.
5. **Route B round-trip gate.** If the unchanged recovered C fails its regression check, Route B stops with a specific `rejected`/`unsupported`/`error` outcome — and a successful compile alone is never shown as a successful repair.
6. **Honest partial state.** Cancel a running job: the state becomes `cancelled` only after termination is confirmed, completed evidence is retained, and any later job is blocked until the guest is reset if termination could not be confirmed.
7. **Missing evidence cannot pass.** With one required obligation left `not_run`, the route shows `inconclusive`, not success, and the UI shows the unresolved obligation beside the result.
8. **Diagnostic candidates stay labelled.** A rejected candidate can be exported but is marked as a diagnostic candidate, and no export implies production approval.
9. **Loopback is still protected.** A cross-origin page attempting a state-changing POST to `127.0.0.1` is rejected; a missing or reused bootstrap token is rejected.
10. **Public/client boundary holds.** With no client authorization recorded, an attempt to send client material to the provider is refused and logged.

Verification Ladder

Evidence required per gate. A candidate needs every gate that applies to it; a missing required obligation prevents `success_within_scope`.

Gate	Required evidence	What it establishes
V0 identity/preconditions	Hashes, approval, environment, complete required inputs	The checks refer to the intended frozen artifacts
V1 structure/placement	Parse/load map, edit manifest, instruction and transfer review, permissions, dependencies	The candidate meets the supported structural profile
V2 interface/compatibility	Reviewed signature, stack/register obligations, loader execution, relevant metadata	Evidence for the supported ABI/runtime profile
V3 concrete behaviour	Original reproducer, invalid-input oracle, boundary tests, valid suite, side-effect observations	The specified test cases satisfy their contract
V4 bounded symbolic security	Candidate-specific model, input domain, memory-access predicates,	The required property holds within the **explicitly reported**

Gate	Required evidence	What it establishes
	exploration and solver accounting	model and domain, if complete
V5 reproducibility	Final hashes, versions, replay inputs, independent teammate replay	Evidence can be reproduced under recorded conditions

V4 is function-level under entry preconditions on the **actual final candidate ELF** (including any redirect and added code), never "whole-program verified". Report all active, terminal, errored, unconstrained, deferred, and resource-pruned states; a path budget is a computational guard, not permission to discard reachable states. Library hooks must be enumerated and justified, and a hook may not replace the repaired function, guard, trampoline, or vulnerable operation with assumed-safe behaviour.

AI Checks

The product uses AI, so these apply to the DeepSeek integration. Model explanations are never evidence.

- **Direct prompt:** a valid public-case request returns a schema-valid candidate bound to the correct original/contract/analysis hashes and the route's allowed target.
- **Bad/indirect prompt:** a candidate containing executable commands, permission requests, an out-of-contract target, or instructions embedded in binary/derived-content strings is rejected by local validation, and the embedded text never alters scope, tools, or permissions.
- **Auth-required prompt:** a request that would send client material externally without recorded client authorization is refused, before any provider call is made.
- **Failure case:** provider timeout, quota exhaustion, empty content, truncated JSON, and schema-invalid JSON each produce a typed failure with diagnostics preserved, no fabricated candidate, and at most the configured retry.
- **Budget exhaustion:** exceeding the per-route call cap or the per-job spending cap stops that route with an explicit reason; unknown usage is reported as unavailable, not zero.
- **Tool/action check:** the model is offered candidate generation only; no execution, write, deletion, network, or credential-bearing action is reachable, and attempted ones are blocked.
- **Data check:** API keys, session tokens, private logs, evaluator source, the reference fix, and confidential derived content must not appear in model output, prompts, events, or traces.

Replay

Replay validates candidate bytes and the recorded environment, then reruns validation. It does not require reproducing identical stochastic model output. Record explicitly whether the replay starts from the original to reproduce patch generation, or from a frozen candidate to reproduce its evidence — the two establish different things. A second teammate must complete this without access to original source.